

Implementation Guide Artifact-Derived Rules (XSD → OWL Layer)

Implementation Guide: Artifact-Derived Rules (XSD → OWL Layer)

1. Purpose of This Layer

The **artifact-derived rules layer** translates *normative constraints embedded in XML Schemas (XSD)* into **explicit semantic rules** that can be reasoned over, cited as evidence, and explained to users.

This layer answers questions such as:

- Is an element **mandatory or optional**?
- Can an entity occur **once or multiple times**?
- Is a relationship **required by the standard**?

without contaminating:

- the **minimal core ontology** (`nits-core`), or
- the **conceptual standard ontology** (e.g. `netex.ttl`).

Key principle: XML Schemas are normative artifacts. Their constraints must be *interpreted semantically*, not ignored and not copied verbatim into conceptual ontologies.

2. Architectural Role

The artifact-derived rules layer sits **between raw schemas and logical reasoning**.

```
1 NeTEx XSD files
2   ↓ semantic interpretation
3 Artifact-Derived Rules Ontology (OWL)
4   ↓
5 GraphDB Reasoning
6
7
```

Plain Text

Characteristics:

- **Standard-specific** (e.g. NeTEx only)
- **Version-specific** (one layer per schema version)
- **Replaceable** when schemas evolve
- **Traceable** back to concrete artifacts

3. What This Layer Is — and Is Not

✓ What It Is

- A semantic interpretation of **XSD constraints**
- A rule layer expressing **normativity** (mandatory/optional)
- A machine-checkable and human-explainable representation

 What It Is Not

- Not a copy of XSD in RDF
- Not XML validation logic
- Not part of `nits-core`
- Not part of the conceptual NeTEx ontology

4. Source Material: What to Extract from XSD

Only extract **semantically meaningful constraints**.

Primary XSD Features to Interpret

	 XSD Feature	 Semantic Meaning	 Example Question Enabled
1	<code>minOccurs</code>	Mandatory vs optional	"Is X mandatory?"
2	<code>maxOccurs</code>	Cardinality / multiplicity	"Can X occur more than once?"
3	Required attributes	Mandatory property	"Must X specify Y?"
4	Containment structure	Dependency	"Does X require Y to exist?"

What Not to Extract

- Grouping constructs
- Abstract syntax helpers
- Ordering sequences (`xs:sequence` alone)
- Technical XML inheritance (`extension` , `restriction`) unless they affect normativity

5. General Modeling Strategy (Important)

Separation of Concerns

	 Layer	 Responsible For
1	XSD	Data validation

2	Conceptual ontology	Meaning of concepts
3	Artifact-derived rules	Normative constraints

Guiding Rule

Translate “how the schema restricts instances” into “what the standard requires conceptually”.

6. Example 1: Optional vs Mandatory Element

XSD Fragment (NeTEx-like)

</> XML

```
1 <xs:element name="ServiceJourneyPattern"
2           type="ServiceJourneyPatternType"
3           minOccurs="0"
4           maxOccurs="unbounded"/>
```

Semantic Interpretation

- `minOccurs="0"` → element is optional
- `maxOccurs="unbounded"` → element is repeatable

OWL Representation (Artifact-Derived Rules Ontology)

Turtle

```
1 @prefix netex: <https://netex.org.uk/ontology/netex#> .
2 @prefix rule:  <https://napcore.eu/ontology/rules#> .
3 @prefix xsd:   <http://www.w3.org/2001/XMLSchema#> .
4
5 rule:ServiceJourneyPattern_Optional
6   a rule:OptionalElementRule ;
7   rule:appliesTo netex:ServiceJourneyPattern ;
8   rule:minOccurs 0 ;
9   rule:maxOccurs "unbounded" ;
10  rule:derivedFromSchema "ServiceJourney.xsd" ;
11  rule:derivedFromXPath  "/Service/ServiceJourneyPattern" .
12
```

What This Enables

- Reasoner can conclude: *ServiceJourneyPattern* is optional in NeTEx core
- Explanation layer can cite the exact schema source

7. Example 2: Mandatory Containment

XSD Fragment

XML

```
1 <xs:element name="Timetable"
2           type="TimetableType"
3           minOccurs="1"
4           maxOccurs="1"/>
```

Semantic Interpretation

- A related parent concept **requires exactly one timetable**

OWL Representation

</> Turtle

```
1 rule:Service_requires_Timetable
2   a rule:MandatoryContainmentRule ;
3   rule:subject netex:Service ;
4   rule:object netex:Timetable ;
5   rule:minOccurs 1 ;
6   rule:maxOccurs 1 ;
7   rule:derivedFromSchema "Service.xsd" .
```

Why this rule exists (and why it does not imply similar rules for every NeTEx module)

This rule is included as a **concrete NeTEx example** to illustrate *when* an artifact-derived rule is justified and *why* it can be expressed safely.

It does *not* imply that every NeTEx module (e.g. `StopPlace`, `Accessibility`, `Facilities`) must now have corresponding rules.

The decision to add this rule follows a strict, question-driven rationale:

1. A realistic user question exists

"Does a Service require a Timetable in NeTEx?"

1. The answer is clearly and unambiguously encoded in the XSD, not merely in prose or profiles.
2. The constraint is structurally normative

It applies universally, not conditionally or context-dependently.

Only when **all three conditions are met** does an element-level constraint deserve to become an artifact-derived rule.

In contrast, many NeTEx modules (such as `StopPlace`) contain numerous elements that are:

- optional by design,
- profile-dependent,
- context-specific,
- or semantically descriptive rather than normative.

For such modules, the *correct* outcome of a review is often **no artifact-derived rules at all**. This is a valid, expected, and architecturally sound result.

Important principle: Reviewing a NeTEx module is a *filtering activity*, not a rule-generation obligation.

The absence of rules after review indicates correct discipline, not incompleteness.

Resulting Reasoning

- System can answer: *Yes, a Service must have a Timetable*
 - Evidence can be shown directly to the user
-

8. Example 3: Required Attribute

XSD Fragment

</> XML

```
1 <xs:attribute name="id" type="xs:string" use="required"/>
```

Semantic Interpretation

- Instances must specify an identifier

OWL Representation

</> Turtle

```
1 rule:Service_requires_ID
2   a rule:MandatoryAttributeRule ;
3   rule:appliesTo netex:Service ;
4   rule:attributeName "id" ;
5   rule:derivedFromSchema "CommonTypes.xsd" .
6
7
```

9. Naming, Versioning, and Scope

One File or Multiple Files? (Critical Design Decision)

Artifact-derived rules **MUST** be separated per standard and per standard version.

This is not an implementation convenience, but a **fundamental architectural requirement**.

✓ Correct Pattern (Required)

- One artifact-derived rules ontology per standard
- One rules ontology per schema version or release

Example:

 Plain Text

```
1 netex-artifact-rules-v1.1.ttl
2 netex-artifact-rules-v1.2.ttl
3
4 siri-artifact-rules-v2.0.ttl
5 opra-artifact-rules-v1.0.ttl
6
7
```

Each file:

- references **only one standard**
- interprets **only that standard's artifacts**
- is aligned implicitly via the standard ontology (not the core)

✗ Incorrect Pattern (Must Be Avoided)

- One global rules file containing rules for NeTEx, SIRI, OpRa, etc.
- Mixing constraints from different standards in the same rules ontology
- Sharing rule individuals across standards

This pattern breaks:

- provenance (which standard enforces this?)
- governance (who owns the rule?)
- versioning (which schema version does it apply to?)
- reasoning scope control

Why Rules Cannot Be Shared Across Standards

Even when standards appear to express similar constraints (e.g. "exactly one timetable"):

- they are **normatively independent**
- they can evolve differently
- they may apply in different contexts

Therefore:

Rules are owned by standards, not by the core, and not by the overall system.

Relationship Between Rules, Core, and Standard Ontologies

The layering is strictly as follows:

Plain Text

```

1  nits-core                (abstract meaning, no rules)
2      ▲
3      | alignment
4      |
5  netex ontology          (conceptual NeTEx semantics)
6      ▲
7      | semantic interpretation
8      |
9  netex artifact-rules    (normative constraints derived from XSD)
10
11

```

Rules:

- refer to **standard ontology concepts** (e.g. `netex:ServiceJourneyPattern`)
- never refer directly to `nits-core` concepts
- influence reasoning **only when that standard is active**

Version Coupling

Each artifact-derived rules file must be coupled to:

- a specific XSD release
- a specific NeTEx (or other standard) version

Rules are **never modified in place** across versions.

If a schema changes:

- create a new rules file
- keep old versions for traceability

Loading Scope

Artifact-derived rules are loaded:

- **only when reasoning about that specific standard**, and
- **only when answering normative or constraint-based questions**

They are **never loaded** during:

- initial concept discovery
- standard-agnostic routing

10. How This Layer Is Used in the QA Pipeline

1. User asks normative question
2. Core ontology identifies abstract concept
3. Alignment activates NeTEx

4. Artifact-derived rules are loaded
 5. Reasoner evaluates constraints
 6. Explanation cites schema-derived rule
-

11. Governance and Maintenance Rules

- Never edit rules without referencing an artifact change
 - Every rule must cite its schema source
 - Removing a rule requires justification
 - Use competency questions to validate rule impact
-

12. Common Anti-Patterns (Avoid These)

✗ Copying XSD mechanically into OWL ✗ Encoding XML syntax in the core ontology ✗ Hard-coding mandatory/optional logic in application code ✗ Ignoring schema constraints in favor of prose text

13. Final Takeaway

Artifact-derived rules are what allow your system to answer “Is this mandatory?” truthfully, repeatably, and explainably.

Without this layer:

- reasoning floats free of standards
- answers lose evidential grounding
- trust erodes quickly

This layer is therefore **non-optional** for a production-grade standards helpdesk.

14. Competency Questions for NeTEx Artifact-Derived Reasoning

Purpose of Competency Questions

Competency questions define **what the NeTEx knowledge layer must be able to answer**, independently of how it is implemented. They serve as:

- acceptance criteria for the NeTEx ontology, alignments, and artifact-derived rules
- guards against semantic over- or under-interpretation
- regression checks when schemas, rules, or alignments evolve

Competency questions are **not implementation rules**. They do **not** imply that every question requires a corresponding artifact-derived rule.

Scope of NeTEx Competency Questions

These competency questions apply to the **combination** of:

- `nits-core`
- NeTEx conceptual ontology
- NeTEx ↔ NITS alignments
- NeTEx artifact-derived rules (where justified)

They explicitly test **behavioral capability**, not schema coverage.

A. Service and Timetable Competency Questions

CQ-NETEX-SVC-01

Can the system determine whether a NeTEx `Service` requires a `Timetable`?

- Expected behavior: *Yes*
- Evidence source: NeTEx XSD (structural constraint)
- Requires artifact-derived rule: ✓ `Service_requires_Timetable`

CQ-NETEX-SVC-02

Can the system explain *why* a `Timetable` is required for a `Service`, citing a concrete NeTEx artifact?

- Expected behavior: Explanation references schema file and path
 - Requires artifact-derived rule: ✓
-

B. Journey Pattern Competency Questions

CQ-NETEX-JRP-01

Can the system determine whether a `ServiceJourneyPattern` is optional or mandatory in core NeTEx?

- Expected behavior: *Optional*
- Evidence source: `minOccurs="0"` in schema
- Requires artifact-derived rule: ✓ `ServiceJourneyPattern_Optional`

CQ-NETEX-JRP-02

Can the system distinguish between *repeatable* and *singular* journey patterns?

- Expected behavior: Recognize unbounded cardinality
 - Requires artifact-derived rule: ✓ (same rule as above)
-

C. StopPlace Competency Questions (Illustrating “No-Rule” Outcomes)

CQ-NETEX-STOP-01

Can the system correctly report that no universally mandatory sub-structure exists for `StopPlace` in base NeTEx?

- Expected behavior: No mandatory containment is inferred
- Evidence source: Absence of mandatory structural constraints in XSD
- Requires artifact-derived rule:  No

CQ-NETEX-STOP-02

Can the system avoid falsely asserting mandatory accessibility, facilities, or sub-stop requirements for `StopPlace`?

- Expected behavior: *No mandatory claims made*
- Requires artifact-derived rule:  No

These competency questions explicitly validate that **not adding rules** is the correct outcome for certain modules.

D. Identity and Identification Competency Questions

CQ-NETEX-ID-01

Can the system determine whether a `Service` instance must have an identifier?

- Expected behavior: *Yes*
- Evidence source: required attribute in schema
- Requires artifact-derived rule:  `Service_requires_ID`

CQ-NETEX-ID-02

Can the system explain that requirement with reference to the NeTEx XSD?

- Expected behavior: Schema reference is included in explanation
-

E. Standard Activation Competency Questions

CQ-NETEX-ACT-01

When a question concerns planned transport structures (services, timetables, journey patterns), does the system activate NeTEx automatically?

- Expected behavior: NeTEx activated via alignment from `nits-core`

CQ-NETEX-ACT-02

When a question concerns operational responsibility or real-time situations only, does the system avoid activating NeTEx?

- Expected behavior: NeTEx remains inactive
 - Requires artifact-derived rule:  No
-

How Competency Questions Are Used

For each NeTEx release or update:

1. Review competency questions
2. Verify existing rules satisfy required questions
3. Confirm that unanswered questions **do not implicitly require new rules**

4. Add artifact-derived rules **only** when a competency question cannot be satisfied otherwise
-

Important Constraint

Competency questions define expected knowledge behavior — not required rule count.

A valid outcome of a competency question review may be:

- “Satisfied by existing rules”
- “Satisfied without artifact-derived rules”
- “Cannot be satisfied — rule addition justified”

All three are correct outcomes.

Final Principle

Competency questions ensure that artifact-derived rules exist only where they create real semantic value, and nowhere else.

They are the primary mechanism that prevents schema-mirroring and semantic overreach.

15. Competency Questions for SIRI Artifact-Derived Reasoning

Purpose and Scope

This section defines **competency questions specific to SIRI (Service Interface for Real-time Information)**. These questions specify what the system must be able to answer when dealing with **real-time information exchange**, independently of NeTEx planning structures and independently of how SIRI messages are technically encoded.

SIRI competency questions intentionally focus on:

- real-time data semantics
- message intent and scope (e.g. estimation vs situation)
- activation boundaries between NeTEx (planning) and SIRI (real-time)

They apply to the **combination** of:

- `nits-core`
- SIRI conceptual ontology
- SIRI ↔ NITS alignments
- SIRI artifact-derived rules (only where XSD imposes semantic constraints)

A. Real-Time vs Planned Information Boundary

CQ-SIRI-RT-01

Can the system distinguish between *planned* timetable information (NeTEx) and *real-time* information (SIRI)?

- Expected behavior: Planned data activates NeTEx; real-time updates activate SIRI
- Requires artifact-derived rule: ✗ No

- Rationale: This distinction is semantic and alignment-based, not XSD-based

CQ-SIRI-RT-02

Can the system avoid activating SIRI when a question concerns only static or planned transport data?

- Expected behavior: SIRI remains inactive
 - Requires artifact-derived rule: ✗ No
-

B. Estimated vs Actual Data Competency Questions

CQ-SIRI-EST-01

Can the system determine whether a SIRI message represents *estimated* (predicted) values rather than planned ones?

- Expected behavior: Classification as real-time estimation
- Requires artifact-derived rule: ✗ No
- Rationale: This distinction is encoded in message semantics, not structural constraints

CQ-SIRI-EST-02

Can the system explain that estimates in SIRI override but do not redefine NeTex planning data?

- Expected behavior: Explanation references semantic layering (NeTex + SIRI)
 - Requires artifact-derived rule: ✗ No
-

C. Situation and Disruption Competency Questions

CQ-SIRI-SIT-01

Can the system determine whether a SIRI message represents a *Situation* (e.g. disruption, incident)?

- Expected behavior: Correct classification as real-time situation
- Requires artifact-derived rule: ✗ No

CQ-SIRI-SIT-02

Can the system avoid interpreting every SIRI update as a disruption?

- Expected behavior: Only SituationExchange messages introduce disruptions
 - Requires artifact-derived rule: ✗ No
-

D. Mandatory Message Structure Competency Questions (Where Rules May Apply)

CQ-SIRI-MSG-01

Can the system determine whether a SIRI real-time delivery message must contain at least one payload element?

- Expected behavior: Mandatory payload existence is recognized
- Evidence source: SIRI XSD structural constraint
- Requires artifact-derived rule: ✔ (only if enforced by XSD)

CQ-SIRI-MSG-02

Can the system explain that requirement with reference to the SIRI schema?

- Expected behavior: Explanation cites SIRI XSD artifact

These are examples where **SIRI artifact-derived rules may legitimately exist**, but only when structural constraints have clear semantic impact.

E. Standard Interaction and Activation Competency Questions**CQ-SIRI-ACT-01**

When a question concerns delays, cancellations, or predictions, does the system activate SIRI automatically?

- Expected behavior: SIRI activated via alignment to `nits:RealTimeInformation`

CQ-SIRI-ACT-02

When a question concerns responsibilities or governance of disruptions, does the system consider OpRa in addition to SIRI?

- Expected behavior: SIRI + OpRa both activated where aligned

Key Constraint for SIRI Competency Questions

Most SIRI competency questions are satisfied by semantic alignment, not by artifact-derived rules.

This reflects the nature of SIRI:

- Normativity is primarily semantic and contextual
- XSD structure often serves message framing, not domain obligation
- Over-deriving rules from SIRI XSD would distort meaning

Summary Principle for SIRI

SIRI artifact-derived rules should be rare, precise, and strictly justified by schema-level semantic constraints.

Competency questions ensure that SIRI remains:

- clearly separated from planning standards (NeTEx)
- correctly scoped to real-time semantics
- protected from schema-mirroring overreach

16. Competency Questions for OpRa Artifact-Derived Reasoning**Purpose and Scope**

This section defines **competency questions specific to OpRa (Operating Roles and Actions)**. OpRa competency questions focus on **responsibilities, roles, authorities, and governance**, rather than on data structures or message payloads.

They apply to the **combination** of:

- `nits-core`
- OpRa conceptual ontology
- OpRa ↔ NITS alignments
- OpRa artifact-derived rules (only where XSD semantics justify them)

OpRa competency questions deliberately emphasize *role semantics* and *responsibility attribution*.

A. Role and Responsibility Attribution

CQ-OPRA-ROLE-01

Can the system determine which organisation or actor holds responsibility for a given operational process?

- Expected behavior: Responsibility correctly attributed via OpRa ontology
- Requires artifact-derived rule: ✗ No
- Rationale: Responsibility is semantic, not schema-structural

CQ-OPRA-ROLE-02

Can the system distinguish clearly between roles (e.g. operator, authority, data provider)?

- Expected behavior: Roles are not conflated
 - Requires artifact-derived rule: ✗ No
-

B. Authority and Governance Competency Questions

CQ-OPRA-GOV-01

Can the system determine which authority governs or oversees a given operational responsibility?

- Expected behavior: Authority correctly identified through OpRa semantics
- Requires artifact-derived rule: ✗ No

CQ-OPRA-GOV-02

Can the system avoid assigning governance responsibility where none is explicitly defined?

- Expected behavior: No implicit governance inferred
 - Requires artifact-derived rule: ✗ No
-

C. Interaction with Other Standards

CQ-OPRA-INT-01

When a question concerns accountability for real-time information or disruptions, does the system activate OpRa alongside SIRI?

- Expected behavior: OpRa + SIRI both activated via alignment
- Requires artifact-derived rule: ✗ No

CQ-OPRA-INT-02

When a question concerns planning structures only, does the system avoid activating OpRa?

- Expected behavior: OpRa remains inactive
- Requires artifact-derived rule: ✗ No

D. Structural Constraints (Rare, but Possible)

CQ-OPRA-STR-01

Can the system determine whether OpRa requires at least one role assignment in an operational context (only if such a constraint exists in XSD)?

- Expected behavior: Structural requirement recognized only if encoded in schema
- Requires artifact-derived rule: ✔ (only if XSD enforces it)

Most OpRa competency questions intentionally **do not require artifact-derived rules**, reflecting the conceptual nature of the standard.

Summary Principle for OpRa

OpRa competency questions validate semantic responsibility modeling, not structural schema obligations.

Artifact-derived rules for OpRa should therefore be:

- extremely rare
- strictly justified by schema-level mandates
- absent unless clearly required

17. Quality Checks (QC) for OpRa Artifact-Derived Reasoning

Purpose of Quality Checks

Quality Checks (QC) ensure that the **OpRa implementation remains correct, minimal, and semantically faithful** as the system evolves. While *competency questions* define **what the system must be able to answer**, QC define **how to verify that the OpRa layer is implemented and maintained correctly**.

These checks are intended for:

- ontology engineers
- reviewers and maintainers
- governance and audit processes

They complement, but do not replace, competency questions.

QC-OPRA-01: Separation of Concerns

Check: OpRa artifact-derived rules (if any) must be clearly separated from:

- the minimal core ontology (`nits-core`)
- the OpRa conceptual ontology

Acceptance criteria:

- No OpRa artifact-derived rule references `nits-core` classes directly
- No structural rules are embedded in the conceptual OpRa ontology

QC-OPRA-02: Justification of Artifact-Derived Rules

Check: Every OpRa artifact-derived rule (if present) must be justified by a concrete XSD constraint.

Acceptance criteria:

- Each rule includes `derivedFromSchema` (and `derivedFromXPath` where applicable)
- Removal of the rule would make at least one OpRa competency question fail

If no such justification exists, the rule must not exist.

QC-OPRA-03: Rarity of Structural Rules

Check: OpRa artifact-derived rules must remain **exceptional** rather than the norm.

Acceptance criteria:

- The OpRa artifact-rules ontology is empty or near-empty by default
- Presence of many structural rules triggers mandatory review

This QC explicitly validates that *absence of rules* is the expected steady state for OpRa.

QC-OPRA-04: Semantic, Not Structural, Responsibility Modeling

Check: Responsibility and role semantics must be expressed through the OpRa conceptual ontology and alignments, not through XSD-derived rules.

Acceptance criteria:

- Questions of “who is responsible” are answered via OpRa classes and relations
- Artifact-derived rules are not used to infer responsibilities or authority

QC-OPRA-05: Cross-Standard Activation Control

Check: OpRa is activated only when responsibility, authority, or governance semantics are relevant.

Acceptance criteria:

- Purely planning-oriented questions do not activate OpRa
- OpRa is activated alongside SIRI only for accountability-related real-time questions

QC-OPRA-06: Version and Provenance Control

Check: OpRa artifact-derived rules (if any) are version-scoped and traceable.

Acceptance criteria:

- Separate artifact-rules ontology per OpRa schema version
 - Rules are never modified in place across versions
 - Historical rule sets remain available for audit
-

QC-OPRA-07: Negative Validation

Check: The system must actively avoid inferring OpRa semantics that are **not explicitly modeled**.

Acceptance criteria:

- No default responsibilities are inferred
 - No implicit governance relationships are invented
 - Lack of OpRa information results in *“unknown / not specified”*, not assumptions
-

Summary of OpRa Quality Philosophy

For OpRa, correctness is more often demonstrated by what the system does *not* infer than by what it does.

A high-quality OpRa implementation is therefore:

- conservative
 - explicit
 - resistant to over-interpretation
 - stable under schema evolution
-

18. GraphDB Configuration Guide for Artifact-Derived Reasoning

Purpose of This Section

This section explains **how to configure GraphDB to correctly load, separate, and reason over:**

- the minimal core ontology (`nits-core`)
- standard conceptual ontologies (NeTEx, SIRI, OpRa)
- alignment ontologies
- artifact-derived rule ontologies

The goal is to ensure:

- strict separation of concerns
- predictable reasoning scope
- explainability and provenance
- safe cross-standard activation

This section defines the **reasoning and execution side** of the architecture and must be read together with **Section 24 (Artifact Repository and Chunking Pipeline)**, which explains how evidential source material is prepared and supplied to the explanation layer without participating in reasoning.

This guide is **conceptual and architectural**, not vendor-locked scripting.

Fundamental Principle: Named Graph Separation

Every semantic layer **MUST** be loaded into its own named graph.

Never load all ontologies into the default graph.

This is essential to:

- prevent unintended cross-standard inference
 - enable selective activation
 - support auditing and explanation
-

Recommended Named Graph Structure

Core Layer

- **Graph:** `graph:nits-core`
 - **Contents:**
 - `nits-core.ttl`
 - **Always loaded**
-

Standard Conceptual Ontologies

- **Graphs:**
 - `graph:netex`
 - `graph:siri`
 - `graph:opra`
 - **Contents:**
 - respective conceptual OWL ontologies
 - **Loaded on demand**, based on question scope
-

Alignment Ontologies

- **Graphs:**
 - `graph:nits-netex-align`
 - `graph:nits-siri-align`
 - `graph:nits-opra-align`
 - **Contents:**
 - OWL alignment axioms only
 - **Loaded together with their corresponding standard**
-

Artifact-Derived Rule Ontologies

- **Graphs:**
 - `graph:netex-artifact-rules-vX.Y`

- `graph:siri-artifact-rules-vX.Y` (if any)
 - `graph:opra-artifact-rules-vX.Y` (usually empty)
 - **Loaded only when:**
 - a competency question requires normativity, or
 - a user question explicitly asks about mandatory / optional semantics
-

Graph Activation Strategy (Runtime)

The system should activate graphs incrementally:

1. Load `graph:nits-core`
2. Identify candidate standards via alignments
3. Load relevant standard + alignment graphs
4. Load artifact-derived rules **only if needed**

This prevents:

- premature normativity
 - unnecessary reasoning overhead
 - misleading explanations
-

Reasoning Configuration

Recommended reasoning profile:

- OWL-RL or OWL-Horst (GraphDB default)
- Forward-chaining preferred for explainability

Key configuration rules:

- Reasoning **MUST** respect named-graph boundaries
 - No reasoning should be triggered by the default graph
 - Inferencing results must retain graph provenance
-

Negative Reasoning Discipline

GraphDB must be configured so that:

- absence of rules does **not** imply prohibition or obligation
- lack of data yields **"unknown / not specified"**

This is especially critical for:

- OpRa (governance)
 - SIRI (real-time semantics)
-

Provenance and Explanation Support

To support explanation generation:

- Retain graph URIs for inferred triples
- Preserve `derivedFromSchema` metadata on rules
- Never materialize inferred constraints without provenance

This enables explanations such as:

"This requirement is derived from NeTEx Service.xsd, rule X, via artifact-derived reasoning."

Anti-Patterns (Must Be Avoided)

- ✗ Loading all ontologies into the default graph
 - ✗ Mixing standards in a single named graph
 - ✗ Running reasoning before narrowing scope
 - ✗ Persisting inferred normativity without provenance
 - ✗ Treating missing rules as negative constraints
-

Summary Principle for GraphDB Configuration

GraphDB is not just a triple store in this architecture – it is the execution substrate for semantic governance.

Correct behavior depends more on **graph organization and activation discipline** than on rule complexity.

A correct GraphDB configuration therefore guarantees:

- semantic safety
 - predictable inference
 - auditability
 - long-term evolvability
-

19. End-to-End Walkthrough: From User Question to Explained Answer

Purpose of This Walkthrough

This section provides a **concrete, end-to-end walkthrough** showing how all layers defined in this guide work together at runtime:

- minimal core ontology
- standard ontologies (NeTEx / SIRI / OpRa)
- alignments
- artifact-derived rules (selectively)
- competency questions
- GraphDB configuration

The goal is to demonstrate how a single user question is processed, how reasoning scope is controlled, and how an explainable answer is produced.

Example User Question

"Is a timetable mandatory for a public transport service?"

This is a typical *normative* question that users of a standards helpdesk ask.

Step 1: Core Concept Identification

1. The question is parsed and key concepts are identified:

- *service*
- *timetable*
- *mandatory*

1. These are mapped to abstract concepts in `nits-core`, such as:

- `nits:Service`
- `nits:Timetable`

At this point:

- no standard is assumed
 - no normativity is inferred
 - only **abstract meaning** is established
-

Step 2: Standard Discovery via Alignments

Using alignment ontologies, the system determines which standards are relevant:

- (*appropriate abstract service concept in* `nits-core`) → aligned to `netex:Service`
- `nits:Timetable` → aligned to `netex:Timetable`

Result:

- **NeTE**x is activated
- **SIRI** and **OpRa** are *not* activated

GraphDB action:

- load `graph:netex`
 - load `graph:nits-netex-align`
-

Step 3: Competency Question Matching

The system checks whether this question corresponds to a known competency question.

Match found:

- CQ-NETEX-SVC-01

Can the system determine whether a NeTEx Service requires a Timetable?

This tells the system:

- the question is valid and expected
 - normative reasoning is required
 - artifact-derived rules *may* be needed
-

Step 4: Conditional Loading of Artifact-Derived Rules

Because the competency question explicitly concerns *mandatoriness*, the system loads:

- `graph:netex-artifact-rules-vX.Y`

No other artifact-rules graphs are loaded.

This ensures:

- minimal reasoning scope
 - no accidental cross-standard normativity
-

Step 5: Reasoning and Evaluation

Within GraphDB, reasoning proceeds as follows:

- `netex:Service` is examined
- artifact-derived rule `Service_requires_Timetable` is applicable
- rule encodes:
 - `minOccurs = 1`

Logical conclusion:

Yes — a NeTEx Service must have exactly one Timetable.

Step 6: Explanation Assembly

The explanation layer collects provenance:

- applicable rule: `Service_requires_Timetable`
- source schema: `Service.xsd`
- reasoning graph: `graph:netex-artifact-rules-vX.Y`

Resulting explanation (illustrative):

“In NeTEx, a Service is required to have a Timetable.

This requirement is derived from the NeTEx XML Schema (`Service.xsd`), where the Timetable element has `minOccurs = 1`.

The constraint is represented explicitly in the artifact-derived rules layer and was applied according to competency question CQ-NETEX-SVC-01.”

Step 7: Negative Reasoning Validation

The system also verifies what **was not inferred**:

- no claims about responsibility (OpRa not active)
- no claims about real-time updates (SIRI not active)
- no assumptions beyond what is explicitly mandated

This guards against over-interpretation.

Contrast Example: A Non-Normative Question

“Who is responsible for handling service disruptions?”

In this case:

- OpRa is activated
- NeTEx artifact-derived rules are *not* loaded
- no mandatory/optional reasoning is attempted

This illustrates how **the same system behaves very differently** depending on the question type.

End-to-End Success Criteria

This walkthrough demonstrates that the architecture:

- activates only relevant standards
 - loads rules only when justified
 - answers competency questions correctly
 - provides traceable explanations
 - avoids accidental inference
-

Final Takeaway

The end-to-end pipeline is driven by questions, constrained by competency criteria, executed through named graphs, and grounded in artifact-derived evidence.

This is what makes the system both *useful to users* and *defensible to standards bodies*.

20. Operational Checklist for Maintenance, Upgrades, and Audits

Purpose of This Checklist

This operational checklist defines **concrete, repeatable steps** to follow when:

- upgrading NeTEx, SIRI, or OpRa versions
- modifying ontologies or alignments
- adding, changing, or removing artifact-derived rules
- performing audits or external reviews

It translates the architectural principles of this guide into **day-to-day operational practice**.

A. General Pre-Change Checklist (Always Required)

Before making *any* change:

- ☐ Identify **which standard(s)** are affected (NeTEx / SIRI / OpRa)
- ☐ Identify **which layer(s)** are affected:
 - conceptual ontology
 - alignment ontology
 - artifact-derived rules
 - GraphDB configuration
- ☐ Confirm the **current standard version(s)** in use
- ☐ Locate the relevant **competency questions** for the affected standard

Rule: No technical change may proceed without identifying the impacted competency questions.

B. Upgrading a Standard (NeTEx / SIRI / OpRa)

When upgrading to a new schema or specification version:

Step B1: Preserve Existing State

- ☐ Do **not** modify existing artifact-rules ontologies
- ☐ Archive previous versions intact
- ☐ Keep historical GraphDB graphs available for audit

Rule: Standards are versioned by *addition*, never by in-place modification.

Step B2: Review Schema Changes

- ☐ Review change logs and schema diffs
- ☐ Identify new, removed, or changed XSD constraints
- ☐ Ignore purely structural or syntactic refactorings

Focus only on changes that could affect:

- mandatory vs optional semantics
 - cardinality
 - required attributes
-

Step B3: Re-evaluate Competency Questions

For each affected competency question:

- ☐ Is the question still valid?
- ☐ Is the answer unchanged?
- ☐ Does answering it require a *new* artifact-derived rule?
- ☐ Does an existing rule become invalid?

Rule: Artifact-derived rules only change if a competency question changes outcome.

Step B4: Create New Artifact-Rules Ontology (If Needed)

- ☐ Create a **new versioned rules file** (e.g. `netex-artifact-rules-v1.3.ttl`)
- ☐ Copy only rules still justified by schema constraints
- ☐ Remove obsolete rules explicitly (do not comment them out)
- ☐ Update `derivedFromSchema` references

C. Adding or Modifying an Artifact-Derived Rule

Use this checklist **for every single rule**.

- ☐ Which competency question does this rule satisfy?
- ☐ Can the question be answered *without* a rule?
- ☐ Is the rule **strictly derived from XSD**, not prose?
- ☐ Does the rule encode *normativity*, not structure?
- ☐ Is `derivedFromSchema` present?
- ☐ Is `derivedFromXPath` present (where applicable)?

Rule: If any checkbox cannot be ticked, the rule must not be added.

D. Validating “No-Rule” Outcomes

Explicitly record cases where review concludes **no rule is required**.

- ☐ Which competency question was reviewed?
- ☐ Why does it *not* require an artifact-derived rule?
- ☐ Is the behavior already covered by conceptual ontology or alignment?

This step is mandatory for:

- StopPlace (NeTeX)
 - most of SIRI
 - most of OpRa
-

E. GraphDB Configuration Validation

After any change:

- ☐ Verify named-graph separation remains intact
- ☐ Confirm no ontology is loaded into the default graph
- ☐ Test selective graph activation for:
 - purely conceptual questions
 - normative questions
 - governance questions
- ☐ Confirm provenance is retained for inferred triples

F. Regression Testing via Competency Questions

For each affected standard:

- ☐ Execute all relevant competency questions
- ☐ Confirm expected answers are unchanged (unless intentionally modified)
- ☐ Confirm explanations still cite correct schema and rule versions

Rule: Competency questions are the authoritative regression test.

G. Audit and Review Readiness

Before external review or audit:

- ☐ Provide competency question lists
- ☐ Provide artifact-rules ontologies with version history
- ☐ Provide GraphDB named-graph documentation
- ☐ Provide at least one end-to-end walkthrough

If this checklist can be satisfied, the system is **audit-ready**.

Final Operational Principle

Operational discipline preserves semantic discipline.

This checklist ensures that the architecture defined in this guide remains stable, explainable, and defensible as standards evolve.

21. Architecture Summary Diagram

Purpose of This Diagram

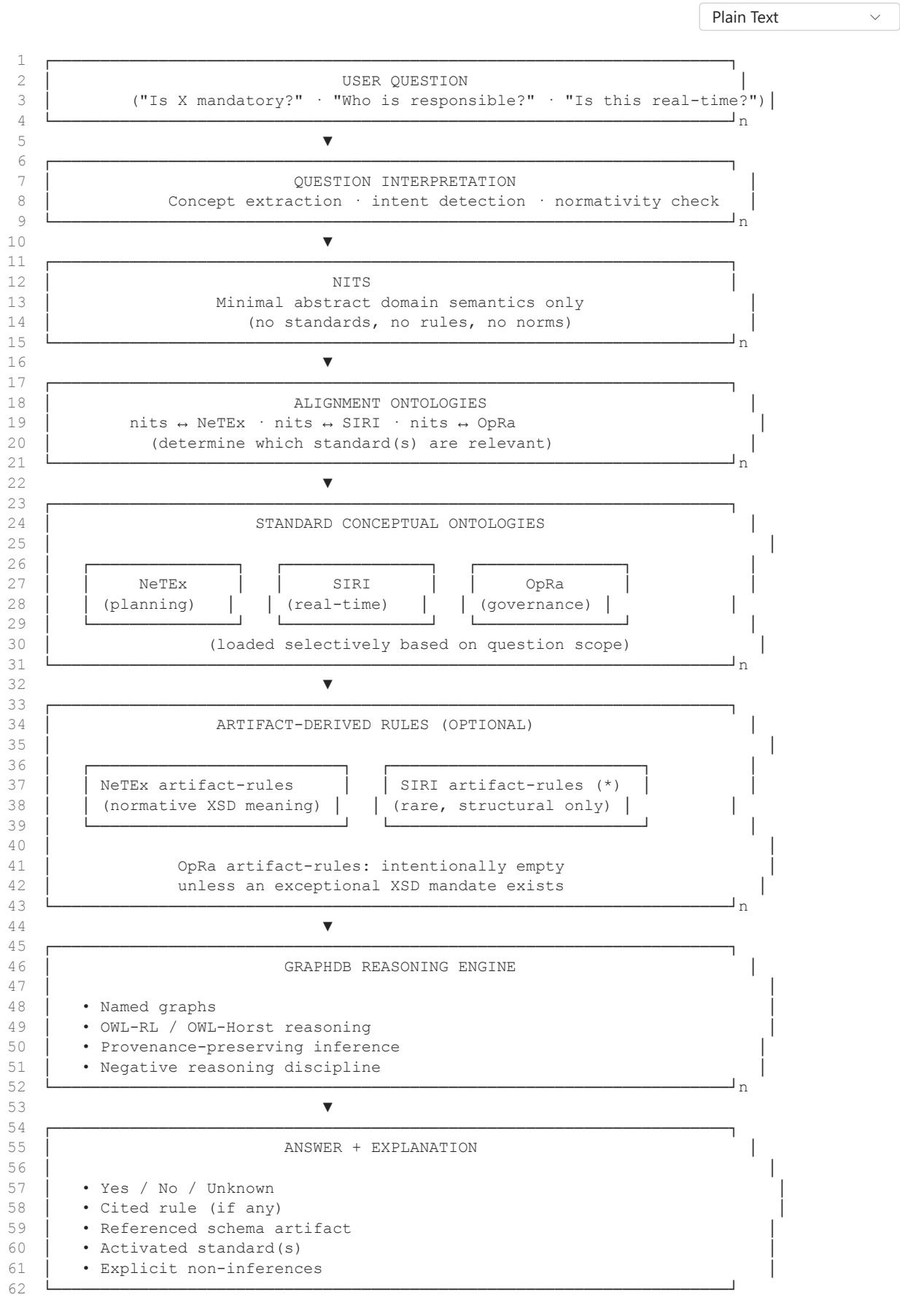
This section provides a **single, compact architecture summary diagram** that visually synthesizes everything defined in this guide:

- semantic layering
- standards separation
- rule isolation
- GraphDB execution model

- question-driven activation

The diagram is **normative**: it reflects the intended steady-state architecture, not an illustrative shortcut.

High-Level Semantic Architecture (Authoritative)



(*) SIRI rules only exist where XSD structure has **clear semantic impact**.

Key Architectural Invariants (Implied by the Diagram)

The diagram enforces the following **non-negotiable constraints**:

1. **No rule exists outside a standard**

Artifact-derived rules always belong to exactly one standard and one version.

1. **No standard activates without alignment**

Standards are never hard-coded or preloaded globally.

1. **No normativity without a question**

Rules are loaded only when a competency question requires them.

1. **No inference without provenance**

Every normative answer can cite an artifact and a rule.

1. **No implication from silence**

Absence of rules or data yields *unknown*, not prohibition or obligation.

How to Use This Diagram

- As a **one-slide architecture overview** for stakeholders
- As a **review checklist** during audits
- As a **design reference** when extending the system
- As a **training artifact** for new team members

If the implementation deviates from this diagram, it is a defect, not a variant.

22. Glossary

Purpose of This Glossary

This glossary defines **terms as they are used in this architecture**, not as they might be used informally elsewhere. The definitions are normative for this guide.

Artifact-Derived Rules

Semantic rules expressed in OWL that are **explicitly and directly derived from normative schema artifacts (XSD)**. They encode mandatory/optional semantics, cardinality, or required attributes, and always cite their source artifact.

Alignment Ontology

An ontology that maps abstract concepts in `nits-core` to standard-specific conceptual ontologies (e.g. NeTeX, SIRI, OpRa), enabling **selective standard activation**.

Competency Question (CQ)

A formally stated question that the system must be able to answer. Competency questions act as:

- acceptance criteria
- regression tests

- justification gates for adding or removing artifact-derived rules

Conceptual Ontology

An ontology that defines the *meaning* of domain concepts within a specific standard, without encoding validation logic or schema structure.

DerivedFromSchema / DerivedFromXPath

Mandatory provenance annotations on artifact-derived rules that identify the exact schema file and (where applicable) XPath expression from which the rule was derived.

Negative Reasoning Discipline

The principle that **absence of data or rules does not imply prohibition or obligation**. In such cases, the correct outcome is *unknown*.

Named Graph

A distinct RDF graph used to isolate semantic layers (core, standards, alignments, rules) to control reasoning scope and preserve provenance.

Normativity

The presence of obligations, requirements, or prohibitions defined by a standard. Normativity is introduced **only** via artifact-derived rules, never via the core ontology.

nits-core

The minimal abstract ontology that captures shared domain meaning, intentionally free of standard-specific semantics, constraints, or rules.

Selective Activation

The runtime mechanism by which only relevant standards (and only relevant rules) are loaded and reasoned over, based on the user's question.

23. Decision Record Template (Artifact-Derived Rules)

Purpose of This Template

This template is used to **formally record decisions** about adding, modifying, or deliberately *not* adding artifact-derived rules.

Using this template is mandatory for:

- schema upgrades
- rule additions or removals
- reviews concluding with a “no-rule” outcome

Decision Record

Decision ID:

ADR-<STANDARD>-<YYYYMMDD>-<SEQ>

Standard:

NeTeX / SIRI / OpRa

Standard Version:

(e.g. NeTeX v1.2)

Schema Artifact(s) Reviewed:

(e.g. Service.xsd)

Schema Version / Release Date:

Competency Question(s):

(list all relevant CQ identifiers)

Decision Type:

- ☐ Add rule
 - ☐ Modify rule
 - ☐ Remove rule
 - ☐ No rule required
-

Evidence and Analysis

Relevant XSD Constraint(s):

- minOccurs / maxOccurs / required attribute / none

Is the constraint structurally normative?

Yes / No

Would removing this rule change the answer to a competency question?

Yes / No

Can the competency question be answered without a rule?

Yes / No

Decision Rationale

(Explain *why* this decision was made. For no-rule outcomes, explicitly justify why no artifact-derived rule is appropriate.)

Resulting Action

- ☐ New artifact-rules ontology created
- ☐ Existing rule retained
- ☐ Existing rule deprecated
- ☐ No changes made

Affected Files / Graphs: -

Review and Approval

Prepared by:

Reviewed by:

Date:

Notes

(Any additional observations, risks, or follow-ups.)

24. Artifact Repository and Chunking Pipeline (GitHub → PostgreSQL)

Purpose of This Section

This section explains **how normative standard source material is ingested, chunked, and stored** from local GitHub repositories into PostgreSQL, and how this artifact store supports (but never pollutes) the semantic architecture defined in this guide.

This pipeline is **foundational infrastructure**, not a semantic layer.

Guiding Constraints (Non-Negotiable)

The artifact ingestion and chunking process follows these strict constraints:

- **Repositories are accessed only locally** (no live GitHub API access at runtime)
- **PostgreSQL is not queried by the reasoner**
- **No reasoning, inference, or validation occurs in PostgreSQL**
- **No ontology or rule is generated mechanically from chunks**

PostgreSQL exists solely to ensure **traceability, provenance, and explainability**.

Source Repositories

The following standards are mirrored locally (read-only):

- NeTeX (schemas, specifications, profiles)
- SIRI (schemas, message definitions, documentation)
- OpRa (schemas and governance material)

Each repository is version-controlled and checked out at explicit tags or commits. The combination of:

- standard name
- repository URL
- commit hash / tag

forms the authoritative identity of a source artifact.

Chunking Pipeline Overview

Plain Text

1

Local GitHub Repository (checked-out version)

2

↓ file discovery

3

Raw Standard Files (XSD, XML, Markdown, HTML)

4

↓ structural parsing

5

Addressable Fragments (file, section, XPath)

6

↓ normalization + metadata enrichment

7

PostgreSQL Artifact Store

What “Chunking” Means (Precisely)

Chunking is **controlled decomposition**, not free-text splitting.

A *chunk* is defined as:

The **smallest addressable fragment** of a standard artifact that can be cited as evidence.

Typical chunk units:

	≡ Artifact Type	≡ Chunk Boundary
1	XSD	element / attribute / complexType (XPath-addressable)
2	XML	definitional section
3	Markdown / HTML	section or subsection
4	Specification text	numbered clause

Every chunk is:

- structurally meaningful
- individually referenceable
- stable under re-chunking of unrelated files

PostgreSQL Storage Model (Conceptual)

Each stored chunk includes, at minimum:

- **standard** (NeTeX / SIRI / OpRa)
- **standard version**
- **repository identifier**

- **commit hash or tag**
- **file path**
- **fragment locator** (XPath, section ID, or offset)
- **raw content** (verbatim)
- **content type** (XSD, prose, schema, etc.)
- **ingestion timestamp**

This makes every chunk:

- reproducible
- auditable
- independently verifiable

Relationship to Ontologies and Rules

The artifact store is used **only upstream** of semantics and in strict coordination with GraphDB configuration rules described in **Section 18 (GraphDB Configuration Guide for Artifact-Derived Reasoning)**.

✓ Used for

- deriving artifact-derived rules
- validating alignment decisions during ontology engineering
- generating human-readable explanations
- evidential citation in QA responses

✗ Explicitly not used for

- OWL reasoning
- constraint checking
- schema validation
- direct SPARQL access

At runtime:

- GraphDB sees **only OWL graphs**, loaded and scoped according to Section 18
- PostgreSQL is consulted **only by the explanation layer**

Example: From Chunk to Explanation

1. PostgreSQL chunk identifies:

- `Service.xsd`
- XPath `/Service/Timetable`
- `minOccurs = 1`
- Ontology engineer creates:
 - `Service_requires_Timetable` (artifact-derived rule)

- Rule records:
 - `derivedFromSchema"
 - `derivedFromXPath`
- At question time, the system:
 - applies OWL reasoning in GraphDB
 - retrieves the matching chunk for explanation

This preserves clear separation between inference and evidence.

Why Repositories Are Local-Only (By Design)

Local access guarantees:

- deterministic builds
- version stability
- audit reproducibility
- independence from external availability

Standards evolution is handled by:

- explicit repository updates
- new chunking runs
- new artifact-rules ontology versions

Never by silent upstream changes.

Architectural Principle Reinforced

The PostgreSQL artifact store is a source of truth for evidence, not for meaning.

Meaning lives in ontologies.

Normativity lives in artifact-derived rules.

Evidence lives in PostgreSQL.

This separation is what keeps the system trustworthy.

End of document.